

Track A: A Scratch Visual Encoder for PushT

Algorithm, Mathematical Objectives, and Experimental Protocol

AdaJEPA experimental branch: `robust-encoder-scratch`

Implementation snapshot: September 27, 2026

Abstract

This document specifies the implemented Track A experiment in the AdaJEPA repository. A task-specific vision transformer is trained from random initialization using masked reconstruction, same-state color translation, and corruption-to-clean reconstruction. Target rendering colors are supplied to a shared decoder, while the encoder is encouraged to retain scene information that is stable across appearance changes. We describe the exact losses, offline four-view dataset, alternating optimization schedule, frozen linear probes, comparison groups, logging, and reproducibility constraints. The implementation has passed local smoke tests; no large-scale training, benchmark improvement, or MPC success gain has yet been established. Experimental recommendations are explicitly separated from implemented behavior.

Repository: <https://github.com/jinu7513/adajepa>

Reviewed base: 66c5f18d00cc627ae35b9b183367f7e510972106

Source of truth: `conf/tracka.yaml` and `tracka/`

Status: implementation and smoke validation; scientific results pending.

Contents

1	Research question and motivation	3
1.1	Why study the visual encoder?	3
1.2	Why these design choices?	3
1.3	Scope and claim boundaries	3
2	Notation and observation model	4
3	Offline four-view dataset	4
3.1	Exactly four views per accepted state	4
3.2	Canonical state and reset verification	5
3.3	Continuous color randomization	5
3.4	Continuous corruption strengths	5
3.5	Splits, sampling and metadata	6
4	Encoder and shared reconstruction decoder	6
4.1	Visible-only bidirectional encoder	6

4.2	Mask-token restoration	7
4.3	Continuous target-RGB conditioning	7
4.4	Shape contract and optional CLS	7
5	Exact training objectives	8
5.1	Masked reconstruction primitive	8
5.2	Clean MAE baseline	8
5.3	Render objective: clean-centered symmetric translation	8
5.4	Corruption objective: noisy input, clean target	9
5.5	Optional global reconstruction and collapse diagnostics	9
6	Optimization algorithm and budget	9
6.1	Four selectable modes	9
6.2	Alternating algorithm	9
6.3	Fairness of the comparison	10
7	Frozen representation evaluation	11
7.1	Physical-state ridge probe	11
7.2	Metrics and their meaning	11
7.3	Test conditions	11
7.4	Nuisance diagnostics and references	12
8	Experimental design and decision rules	12
8.1	Hypotheses	12
8.2	Primary matrix	12
8.3	Staged execution and ablations	13
8.4	What would support or weaken the claim?	13
8.5	Statistical reporting	14
9	Logging, reproducibility and run management	14
10	Implementation evidence and limitations	14
A	Runnable command sequence	15
B	Code-to-method map	16

1 Research question and motivation

1.1 Why study the visual encoder?

PushT planning depends on observing the agent position, block position and orientation, and target geometry. A change in object color or a moderate blur can alter image features without changing the underlying scene arrangement. If the representation moves substantially under such interventions, a predictor or distance-based planning objective may respond to appearance rather than task-relevant geometry. Track A tests this potential failure point at the representation level before modifying planning or dynamics prediction.

The question is: can a PushT-specific encoder trained from scratch preserve linearly accessible scene geometry while becoming less sensitive to controlled rendering nuisance and moderate corruption? Better robustness is a hypothesis, not an assumed consequence of the architecture.

1.2 Why these design choices?

1. **Scratch initialization** isolates what can be learned from this task’s data. Frozen DINOv2 is an external pretrained reference, not a matched-pretraining control.
2. **Paired static rerendering** holds scene state approximately fixed while changing appearance, so aligned image patches have a meaningful correspondence.
3. **Masked reconstruction** requires the representation to support missing scene content. The visible-token encoder and lightweight reconstruction decoder follow the MAE design pattern [1]; our task-specific losses and 50% mask are implementation choices rather than a reproduction of that paper’s full recipe.
4. **Target-color conditioning** gives the decoder an explicit appearance input, reducing the need for the encoder to preserve source color for reconstruction.
5. **A separate corruption objective** avoids requiring reconstruction of an unpredictable noise realization.
6. **Frozen clean-trained probes** test whether the same readout continues to work after visual shift, rather than allowing a new probe to compensate for it.

1.3 Scope and claim boundaries

This branch implements encoder pretraining and representation evaluation. It does not implement Flow-JEPA, flow matching, a new MPC algorithm, test-time adaptation, or a policy. Earlier last-block/last-two-block and multistep AdaJEPA adaptation experiments concern a different training/evaluation mechanism. Here *frozen* means frozen encoder weights during probe fitting, followed by a fixed probe during shifted evaluation. It does not mean the original paper’s Frozen-versus-AdaJEPA planning comparison.

Matching the existing visual token shape is necessary for interface compatibility, but a DINO-trained world-model predictor cannot be assumed to understand a newly trained scratch latent space. End-to-end planning requires a later integration and training protocol.

2 Notation and observation model

Let the available simulator state be

$$s = (a_x, a_y, b_x, b_y, \theta, v_{a,x}, v_{a,y}) \in \mathbb{R}^7. \quad (1)$$

It contains agent and block positions, block angle, and agent velocity. It does not contain block linear/angular velocity or a complete physics snapshot. Let the rendering nuisance be

$$\eta = (c_b, c_a, c_g), \quad c_o \in [0, 1]^3, \quad (2)$$

where RGB vectors describe the block, agent and goal respectively. If R is the renderer and C a corruption operator,

$$x = R(s, \eta), \quad x_{\text{corr}} = C(R(s, \eta), \epsilon). \quad (3)$$

Here ϵ includes random draws; corruption type and strength are recorded as metadata but never supplied to the encoder or decoder.

The informal goal $E_\theta(x) \approx f(s)$ should be read as stability of *visually observable* scene information, not invertibility of the full simulator state. Two states with different velocities may produce the same static image. Velocity probing is therefore auxiliary, not a decisive test of visual sufficiency.

Symbol	Meaning
E_θ, D_ϕ	Scratch image encoder and shared reconstruction decoder
x_0, x_A, x_B, x_c	Clean, two color variants, and corrupted clean image
$P = 196, Q = 768$	Number of patches and RGB pixel values per 16×16 patch
$d = 384, d_D = 192$	Encoder and decoder embedding dimensions
M_i, V_i	Masked and visible patch indices for sample i
Z_i^u	Visible patch representations of view u
B, T	Batch size and total optimizer-update budget
$\lambda_r, \lambda_c, \lambda_g$	Render, corruption and optional global loss weights

Table 1: Core notation. Defaults refer to the checked-in configuration.

3 Offline four-view dataset

3.1 Exactly four views per accepted state

For each selected source state, generate

$$x_0 = R(\bar{s}, \eta_0), \quad x_A = R(\bar{s}, \eta_A), \quad (4)$$

$$x_B = R(\bar{s}, \eta_B), \quad x_c = C(x_0, \epsilon). \quad (5)$$

The notation $\bar{s}$ denotes the verified canonical post-reset state; equalities between separately reset views hold within the configured numerical tolerances. The corrupted image is computed directly from the stored clean render. It does not start from a random-color image. Stored images are uint8 PNGs; normalization is applied in the loader.

The source pool is explicitly selected through `dataset.source_path` and `dataset.source_splits`; the latter defaults to `[train]`. The generator reads state tensors and valid sequence lengths, optionally filters shape metadata to T objects, and rerenders all views. It does not use the planning-target file `val_T/plan_targets.pkl` for pretraining.

3.2 Canonical state and reset verification

The current reset path performs one 0.01-second physics step after assigning the input state. Let s^{src} be the source state and

$$\bar{s} = \text{Reset}(s^{\text{src}}, \eta_0), \quad s^q = \text{Reset}(s^{\text{src}}, \eta_q). \quad (6)$$

Every color view resets from s^{src} , not from $\bar{s}$, to avoid an additional unintended physics advance. Compare each returned state to both s^{src} and $\bar{s}$. Use coordinate-wise maximum absolute error for each position/velocity group and the wrapped angular difference

$$\delta_\theta = \text{atan2}(\sin(\theta - \theta_{\text{ref}}), \cos(\theta - \theta_{\text{ref}})). \quad (7)$$

Reject the complete sample if any tolerance fails. Probe labels use $\bar{s}$, which corresponds to the rendered observation.

Field	Source tolerance	Canonical tolerance
Agent position	5.0	10^{-4}
Block position	5.0	10^{-4}
Block angle (rad)	0.1	10^{-4}
Agent velocity	10^{-3}	10^{-4}

Table 2: Current candidate reset tolerances. Positions use simulator coordinates; velocity uses simulator velocity units. Inspect rejection rates before real training.

Rejection can bias the accepted data toward slower or non-contact scenes. The generation report stores per-field maximum errors and rejected counts by split and trajectory segment; the rejection process must be reported, not hidden by arbitrarily widening thresholds.

3.3 Continuous color randomization

For each object, sample HSV independently before joint rejection:

$$h \sim U(0, 1), \quad s_{\text{HSV}} \sim U(0.45, 0.90), \quad v_{\text{HSV}} \sim U(0.40, 0.85). \quad (8)$$

Convert to RGB and round to uint8. All rejection distances are Euclidean distances in the *actual quantized RGB divided by 255*, rather than naive distances in HSV. The default minimum distances are 0.4 from white background, 0.3 between different objects in one view, and 0.15 between corresponding objects across views. Both A and B change all three object colors relative to clean; B also differs from A. At most 10,000 proposals are attempted before an explicit error.

The default clean RGB values, ordered block/agent/goal, are (119, 136, 153), (65, 105, 225) and (144, 238, 144). Both pre-quantization sampled colors and actual renderer RGB are saved. Separate random streams make assignments independent of scene geometry in the sampling design; a fixed finite dataset can still exhibit accidental correlations, which probes and held-out trajectories help expose.

3.4 Continuous corruption strengths

Corruption types are approximately balanced over selected candidates using a shuffled schedule. Rejection may slightly alter the accepted distribution. Each accepted state stores one fixed corruption realization, reused across epochs. Canonical salt/pepper probability means expected *selected* pixel fraction; a pixel that was already white/black may not visibly change.

Type	Training range	Exact semantics
Gaussian	$\sigma \sim U(1, 8)$	Independent zero-mean channel noise in uint8 intensity units; clip to $[0, 255]$ and cast to uint8.
Salt/pepper	$p \sim U(0.001, 0.02)$	One uniform draw per pixel; probability $p/2$ of white, $p/2$ of black, otherwise unchanged.
Blur	$\sigma \sim U(0.3, 1.2)$	Gaussian sigma in image pixels, kernel $\max(3, 2 \text{ round}(3\sigma) + 1)$; zero sigma is identity.

Table 3: Candidate defaults requiring a visual sanity check on representative source states.

Historical snp1/snp5 separately sample salt and pepper positions with replacement and retain those original semantics. dark remains available to existing evaluation code but is not one of Track A’s default training corruptions.

The generator first exports deterministic contact sheets, including corruption strength endpoints and a 50% spatial mask. Full generation requires `generation.ranges_reviewed=true`. This acknowledges image inspection; it is not an automated proof that every mask retains every object.

3.5 Splits, sampling and metadata

Trajectory IDs are assigned once to approximately 70/15/15% train/validation/test. At least one validation and one test trajectory are reserved, so tiny smoke datasets need not have exact percentages. Default frame stride is five raw frames; state caps are optional. Sampling and split assignments are stored, never recomputed from directory ordering. All views of a state stay in the same split.

For valid sequence length L and raw timestep t ,

$$u = t / \max(L - 1, 1), \quad \text{segment}(u) = \begin{cases} \text{early}, & 0 \leq u < 1/3, \\ \text{middle}, & 1/3 \leq u < 2/3, \\ \text{late}, & 2/3 \leq u \leq 1. \end{cases} \quad (9)$$

Middle-segment difficulty is an empirical question, not an assumption. The manifest records source directories, selected IDs/timesteps, seeds, caps, color/corruption configuration, schema version, metadata hash and code provenance. Integrity checks verify counts, image paths, verified states and disjoint trajectory sets.

4 Encoder and shared reconstruction decoder

4.1 Visible-only bidirectional encoder

Normalize input pixels as $\tilde{x} = x/127.5 - 1$. A 224×224 RGB image yields $P = (224/16)^2 = 196$ patches, each containing $Q = 16^2 \cdot 3 = 768$ scalars. Let $\pi_p(\tilde{x}) \in \mathbb{R}^Q$ be patch p . Its input token is

$$t_p = W_E \pi_p(\tilde{x}) + b_E + e_p. \quad (10)$$

At mask ratio $r = 0.5$, select $|V| = \lfloor P(1 - r) \rfloor = 98$ visible patches by a random permutation. Gather both patch embeddings and their original positional embeddings using the same `ids_keep`. Do not renumber visible spatial positions.

The encoder uses six pre-normalized transformer blocks with six attention heads, embedding width 384, feed-forward width $4d$, GELU and zero dropout, followed by LayerNorm. Self-attention is spatially bidirectional. No causal predictor mask is reused. One mask is shared by clean/A/B during a render update, and by clean/corrupt during a corruption update. Corresponding latent tokens therefore refer to identical positions.

4.2 Mask-token restoration

Project visible latents to width 192, append learned mask tokens, and restore the original patch order using `ids_restore`:

$$H_0 = \text{Restore}([W_D Z_V + b_D; m, \dots, m]) + E_D. \quad (11)$$

The shared decoder has two blocks, six heads and a pixel head producing $\hat{X} \in \mathbb{R}^{B \times 196 \times 768}$. It predicts every patch, but the main reconstruction loss is evaluated only on masked positions.

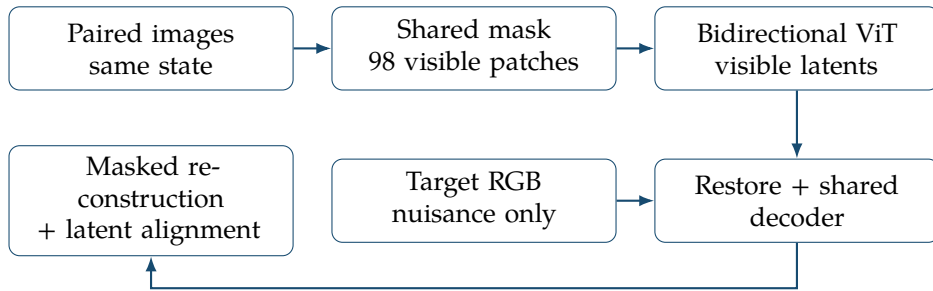


Figure 1: Training information flow. Physical-state labels enter only verification and post-training probes.

4.3 Continuous target-RGB conditioning

For object $o \in \{b, a, g\}$, apply a shared RGB MLP and add a learned type embedding:

$$n_o = f_{\text{RGB}}(c_o) + e_{\text{type}}(o), \quad n_o \in \mathbb{R}^{64}. \quad (12)$$

The default additive path concatenates in block/agent/goal order:

$$n_{\text{global}} = W_n[n_b; n_a; n_g] + b_n \in \mathbb{R}^{192}, \quad H_{0,p} \leftarrow H_{0,p} + n_{\text{global}}. \quad (13)$$

The optional cross-attention path projects the three typed tokens to decoder width, forming $N \in \mathbb{R}^{3 \times 192}$, and applies

$$H' = H + \text{SA}(\text{LN}(H)), \quad (14)$$

$$H'' = H' + \text{CA}(\text{LN}(H'), N, N), \quad (15)$$

$$H_{\text{next}} = H'' + \text{MLP}(\text{LN}(H'')). \quad (16)$$

Additive blocks omit the cross-attention term. The RGB always describes the *target*, including η_0 for denoising. Corruption metadata is excluded.

4.4 Shape contract and optional CLS

Optional CLS processing adds a token internally; the public `forward(images)` still returns patch tokens only. A separate feature API returns CLS when requested. The optional global path projects CLS to decoder width, broadcasts it over decoder positions, and reuses the decoder blocks/head to predict the complete target. It is not a separate independently parameterized decoder network.

Tensor	Default shape
Image	$[B, 3, 224, 224]$
Patch pixels	$[B, 196, 768]$
Full encoder output	$[B, 196, 384]$
Masked encoder output	$[B, 98, 384]$
Decoder visible projection	$[B, 98, 192]$
Restored decoder tokens	$[B, 196, 192]$
Predicted patch pixels	$[B, 196, 768]$

Table 4: Dimensions are configurable; these values describe the default contract.

5 Exact training objectives

5.1 Masked reconstruction primitive

Let $m_{ip} = 1$ for a masked patch and zero otherwise. For source view u and target view v , define

$$\ell(u \rightarrow v) = \frac{\sum_{i=1}^B \sum_{p=1}^P m_{ip} \left[Q^{-1} \sum_{j=1}^Q (\hat{X}_{ipj}^{u \rightarrow v} - X_{ipj}^v)^2 \right]}{\sum_{i,p} m_{ip}}, \quad \hat{X}^{u \rightarrow v} = D_\phi(Z^u, \eta_v). \quad (17)$$

By default X^v is the patchified image in $[-1, 1]$. With `loss.norm_pix_loss=true`, each target patch is further normalized using its own pixel mean and population variance plus 10^{-6} . This is separate from image normalization. Normalized-pixel predictions do not directly determine RGB without target statistics; their current preview panels are intentionally blank.

5.2 Clean MAE baseline

$$\mathcal{L}_{\text{clean}} = \ell(0 \rightarrow 0). \quad (18)$$

This retains the same conditional decoder architecture but provides the fixed clean colors. It tests whether ordinary task-specific masked reconstruction is sufficient.

5.3 Render objective: clean-centered symmetric translation

For each $q \in \{A, B\}$,

$$\mathcal{L}_{\text{cross},q} = \frac{1}{2} [\ell(0 \rightarrow q) + \ell(q \rightarrow 0)], \quad (19)$$

$$\mathcal{L}_{\text{inv},q} = \frac{1}{B|V|d} \sum_{i,p \in V_{i,k}} (Z_{ipk}^0 - Z_{ipk}^q)^2. \quad (20)$$

Average A and B equally within every render update:

$$\mathcal{L}_{\text{cross}} = \frac{1}{2} (\mathcal{L}_{\text{cross},A} + \mathcal{L}_{\text{cross},B}), \quad (21)$$

$$\mathcal{L}_{\text{r-inv}} = \frac{1}{2} (\mathcal{L}_{\text{inv},A} + \mathcal{L}_{\text{inv},B}), \quad (22)$$

$$\boxed{\mathcal{L}_{\text{render}} = \mathcal{L}_{\text{cross}} + \lambda_r \mathcal{L}_{\text{r-inv}}}. \quad (23)$$

There are four directed reconstructions and three encoded views. Gradients flow through both sides of latent alignment; there is no frozen teacher or stop-gradient. The default $\lambda_r = 1$ is a starting value, not evidence of an optimal tradeoff.

5.4 Corruption objective: noisy input, clean target

$$\mathcal{L}_{\text{denoise}} = \ell(c \rightarrow 0), \quad (24)$$

$$\mathcal{L}_{\text{c-inv}} = \frac{1}{B|V|d} \sum_{i,p \in V_i,k} (Z_{ipk}^0 - Z_{ipk}^c)^2, \quad (25)$$

$$\boxed{\mathcal{L}_{\text{corrupt}} = \mathcal{L}_{\text{denoise}} + \lambda_c \mathcal{L}_{\text{c-inv}}}. \quad (26)$$

The default is $\lambda_c = 1$. The decoder receives η_0 , not corruption type, strength or seed. Only masked clean-target pixels are reconstructed: this is not a full-image denoising loss over every visible pixel. The alignment term additionally encourages corresponding visible representations to agree.

5.5 Optional global reconstruction and collapse diagnostics

For each active directed reconstruction, the CLS path contributes full-patch MSE,

$$\ell_g(u \rightarrow v) = \frac{1}{BPQ} \sum_{i,p,j} (D_{g,\phi}(z_{\text{CLS}}^u, \eta_v)_{ipj} - X_{ipj}^v)^2. \quad (27)$$

Average it over the same active directions and add $\lambda_g \mathcal{L}_g$ to the selected objective. CLS, global reconstruction, and λ_g are disabled/zero by default.

Latent agreement alone admits constant or low-information solutions. Reconstruction provides a competing requirement, but it is not a proof against collapse. Monitor unweighted/weighted losses, latent norms, distances, variance across states and physical probes. A small distance accompanied by a poor state probe is not success.

6 Optimization algorithm and budget

6.1 Four selectable modes

Mode	Objective	Encoder views/update	Decoder passes/update
clean_mae	$\mathcal{L}_{\text{clean}}$	1	1
render_only	$\mathcal{L}_{\text{render}}$	3	4
corruption_only	$\mathcal{L}_{\text{corrupt}}$	2	1
robust_alternating	Alternating render/corruption	3 then 2	4 then 1

Table 5: Default CLS-disabled computation per sampled state. Counts exclude validation and previews.

Each run initializes its own encoder/decoder. “Shared decoder” means shared between objectives within a run, not shared weights between experimental runs.

6.2 Alternating algorithm

For even total budget T , initialize one AdamW optimizer for all trainable parameters. For $t = 0, \dots, T - 1$:

1. Draw B accepted training-state indices uniformly with replacement.
2. Load the corresponding offline views and target RGB; do not load state labels into the training batch.
3. Sample a new shared spatial mask for this update.
4. Set $\mathcal{L}_t = \mathcal{L}_{\text{render}}$ if t is even, otherwise $\mathcal{L}_{\text{corrupt}}$.
5. Zero gradients, compute $\nabla_{\theta, \phi} \mathcal{L}_t$, reject non-finite loss/gradient, clip the global gradient norm to one, and call AdamW once.
6. Set $\text{global_step} = t + 1$, record metrics, and periodically validate/save.

One logical cycle consists of two optimizer updates. Clean MAE is not a third default update. The clean encoding is recomputed at each update; the corrupted batch does not reuse stale features from the preceding render update. Sequential AdamW updates with shared moment estimates are not identical to one update on the sum of both losses.

Parameter	Default
Total optimizer updates	10,000
Batch size	16 states
AdamW learning rate / weight decay	10^{-4} / 0.05
Gradient norm cap	1.0
Precision / LR schedule	float32 / constant
Validation / checkpoint interval	100 updates / 100 updates
Validation cap	4 batches
Image logging interval	100 updates
Scalar logging interval	10 per-objective updates

Table 6: Defaults, not tuned optima. Alternating scalar cadence samples both objectives.

The training “epoch” field is cumulative sampled states divided by dataset size; it does not imply a shuffled full pass. Validation currently uses the first capped set of validation records with fixed mask RNG seed 9871. Its losses are diagnostics, not an exhaustive evaluation of every held-out state. Use the final fixed-budget checkpoint for the primary comparison to avoid test-based checkpoint selection.

6.3 Fairness of the comparison

Match architecture, mask, optimizer, dataset and total update count across modes. With $T = 10,000$, alternating uses 5,000 render plus 5,000 corruption updates. This is an *update-matched* comparison, not a compute-matched one. With CLS off, the average encoded/decoded view counts per state draw are 2.5/2.5 for alternating, versus 1/1 for clean MAE. Log image-pass counts and wall time; a later compute-matched comparison would be a separate protocol. Timing is per process segment after resume.

7 Frozen representation evaluation

7.1 Physical-state ridge probe

Freeze the encoder in evaluation mode. Without masking, mean-pool patch tokens:

$$f_i = P^{-1} \sum_{p=1}^P E_{\theta}(x_i)_p \in \mathbb{R}^{384}. \quad (28)$$

Targets are computed from canonical rendered state:

$$y_i = (a_x, a_y, b_x, b_y, \cos \theta, \sin \theta, v_{a,x}, v_{a,y}) \in \mathbb{R}^8. \quad (29)$$

Fit feature and target means/stds using clean training observations only, flooring each standard deviation at 10^{-6} . For standardized matrices F, Y ,

$$W_{\alpha} = \arg \min_W \|FW - Y\|_F^2 + \alpha \|W\|_F^2 = (F^{\top} F + \alpha I)^{-1} F^{\top} Y. \quad (30)$$

The implementation uses the equivalent dual solve when samples are fewer than feature dimensions. Centering accounts for the intercept. Select $\alpha \in \{0.001, 0.01, 0.1, 1, 10\}$ by clean validation standardized-target MSE. Freeze this readout and its normalization statistics before any shifted test evaluation.

7.2 Metrics and their meaning

For condition c with n test samples, the headline aggregate is

$$\text{SMSE}(c) = \frac{1}{8n} \sum_{i,j} \left(\frac{\hat{y}_{ij}^c - y_{ij}}{\sigma_{y,j}^{\text{train}}} \right)^2. \quad (31)$$

Also report agent-position MSE, block-position MSE and velocity MSE in their original units, per-target R^2 , and wrapped angle MAE:

$$\hat{\theta}_i = \text{atan2}(\hat{y}_{i,\sin}, \hat{y}_{i,\cos}), \quad (32)$$

$$\text{MAE}_{\theta} = \frac{1}{n} \sum_i \left| \text{atan2}(\sin(\hat{\theta}_i - \theta_i), \cos(\hat{\theta}_i - \theta_i)) \right|. \quad (33)$$

Angles are reported in radians and degrees. R^2 is undefined and stored as null when the test target has essentially zero variance; negative R^2 is possible. Very small target variance and near-zero predicted sine/cosine pairs require care.

The implemented clean-to-shift degradation is

$$\Delta(c) = \text{SMSE}(c) - \text{SMSE}(0). \quad (34)$$

Evaluation latent distance is RMS distance between *mean-pooled full-image* features. It is distinct from the training loss on aligned visible patch tokens. Neither raw latent distance nor RGB probe performance alone establishes useful invariance.

7.3 Test conditions

The default grids span interpolation and stronger extrapolation points. Check disjointness if training color bounds are changed. Additional evaluation views are generated separately; they do not alter the four-view training schema. Full/early/ middle/late results use the same clean-trained probe. Rendering is verified against the saved canonical state. The primary evaluation never refits a probe per shift.

Condition	Intervention
default, colorA, colorB	Stored held-out views
redBlock, redAgent, redAnchor	Set only the named object’s RGB to (255, 0, 0); anchor means goal marker
random_inside	Fresh random colors under the training sampling bounds
random_outside	Default saturation [0.91, 1], value [0.86, 0.95]; same distance checks
Gaussian grid	$\sigma \in \{0, 4, 8, 12\}$ in uint8 units
Salt/pepper grid	$p \in \{0, 0.01, 0.02, 0.04\}$
Blur grid	$\sigma \in \{0, 0.6, 1.2, 2\}$ in image pixels

Table 7: Implemented evaluation protocol, default evaluation seed 100. Zero-strength points are identity controls.

7.4 Nuisance diagnostics and references

RGB ridge probes are trained on variable-color A/B views, separately for block, agent and goal. Clean colors alone would give constant targets and an uninformative probe. Lower RGB predictability is favorable only if scene information is preserved; a weak linear probe cannot prove statistical independence.

Corruption-type predictability uses a ridge fit to one-hot labels followed by argmax; it is not a logistic classifier. Report accuracy, uniform chance and a training-majority baseline. Strength regression is fit separately for each type to avoid mixing units; insufficient data is recorded explicitly.

References are random scratch, clean MAE, render-only, corruption-only, alternating, and frozen pretrained DINOv2 [2]. DINO remains evaluation-only. The existing wrapper feeds it 196px images with patch14 to yield 196 tokens, whereas scratch uses 224px with patch16. Preserve and record this preprocessing distinction. DINO’s external pretraining data and model history also prevent a strict equal-data comparison.

8 Experimental design and decision rules

8.1 Hypotheses

H1: color invariance. Render-only reduces color-shift degradation relative to clean MAE without a practically unacceptable loss of clean geometric accuracy.

H2: corruption robustness. Corruption-only improves corruption curves, especially within the trained range, relative to clean MAE.

H3: combined utility. Alternating preserves useful improvements in both families under a fixed update budget, rather than merely shrinking feature norms.

H4: decoder ablation. Cross-attention or a CLS global path may change the geometry/invariance tradeoff; improvement is not presumed.

8.2 Primary matrix

Planned experiment. Fix one reviewed dataset manifest, split seed, sampling ranges and evaluation seed. Train the four objectives with seeds $\{0, 1, 2\}$: 12 independent training runs. Use additive conditioning, no CLS, mask 0.5, $\lambda_r = \lambda_c = 1$, and 10,000 updates. Pair seeds across objectives for comparison. Evaluate a random encoder and frozen DINO reference under the same

trajectory/probe protocol. The current random-reference initialization uses `evaluation.seed`; rerunning it with another seed also changes fresh evaluation draws. Keep that seed fixed for the primary reference rather than pooling incompatible protocols as additional training replicates.

Controlled item	Planned control
Data	Same accepted states, manifest, four stored views and split IDs
Initialization	Same training-seed set for the four learned objectives
Optimization	Same architecture, batch, LR, optimizer, clipping and update count
Checkpoint	Final budget endpoint; no test-based selection
Probe	Same mean pooling, clean-only fitting and validation selection grid
Test randomness	Fixed evaluation seed and grids across compared learned runs
Reporting	Per-condition and per-segment errors, uncertainty across training seeds

Table 8: Proposed primary comparison. These runs have not yet been performed.

8.3 Staged execution and ablations

1. Audit real source states and rejection rates; visually review color/corruption contact sheets before freezing the manifest.
2. Run a two-update smoke job with online W&B on CIRCE and verify live metrics, status, local JSONL, checkpoint and reconstruction image.
3. Run clean MAE and alternating for one seed as an engineering pilot. Use it to detect failures, not to claim a robust scientific result.
4. Execute the preregistered four-objective, three-seed primary matrix.
5. Evaluate all selected checkpoints and the two references using the fixed protocol.
6. Only after the primary matrix, run controlled ablations: zero versus nonzero invariance weights, additive versus cross-attention, CLS/global off versus on, and mask ratio. Keep one changed factor per comparison.

Suggested exploratory grids are $\lambda \in \{0, 0.1, 1\}$ and mask ratio $r \in \{0.25, 0.5, 0.75\}$; these are recommendations, not executed or tuned values. With separate render/corruption weights, vary one while fixing the other before attempting a larger interaction study. Record every attempted setting, not only winners.

8.4 What would support or weaken the claim?

Planned experiment. Report clean errors alongside every shifted error. A method that lowers shift degradation by making clean performance worse is not necessarily useful. Before final testing, choose a domain-meaningful acceptable clean geometric-error increase using training/-validation evidence and record it. No numerical acceptance threshold has yet been established in this implementation.

Evidence favoring the approach would combine stable clean position/angle accuracy, smaller errors under both color and corruption interventions, and consistency across training seeds. Improvement only on trained corruption strengths suggests interpolation robustness, not unrestricted generalization. Poor RGB predictability with poor geometry suggests information loss. Good reconstruction with poor probes suggests that pixel reconstruction alone has not established a useful representation.

8.5 Statistical reporting

The current aggregation script reports mean and sample standard deviation across distinct training seeds, not a confidence interval. With K runs,

$$\bar{m} = K^{-1} \sum_k m_k, \quad s_m = \sqrt{\frac{\sum_k (m_k - \bar{m})^2}{K - 1}}. \quad (35)$$

Three seeds provide a first stability check, not strong statistical power. Frames within a trajectory are correlated. Current per-run metrics are frame-weighted; longer accepted trajectories can contribute more. A future trajectory-balanced or trajectory-bootstrap analysis should resample whole trajectories, not independent frames, and would require retaining/exporting per-sample predictions beyond the current aggregate reports. Do not label existing SD bars as confidence intervals.

9 Logging, reproducibility and run management

Implemented. W&B project defaults to `adajepa_trackA`. Names include objective, conditioning, mask ratio and seed. Training curves use `train/global_step`. Local JSONL is flushed before W&B logging, so scalar records survive service failure. Config, dataset manifest hash, source hash, git SHA and dirty state accompany the run. Per-condition probe tables, color-shift bars and separate strength curves are also recorded; all have local counterparts.

Monitor `render/cross_mae`, `render/invariance`, their weighted form, corruption denoising/invariance counterparts, `clean/mae`, total/global losses, latent norms/variance/distances, gradient norm, LR, sample count, image-pass counts, and elapsed time. Validation diagnostics use separate keys. Training epoch, optimizer step and MPC step are different concepts; this branch produces no MPC success-rate curve.

Checkpoint dictionaries save encoder, shared decoder and optimizer states; update, cycle and sample counters; next objective; RNG states; resolved config; manifest/schema identity; git/source provenance; output path and W&B run ID. Float32 constant-LR training has no AMP scaler or scheduler to resume. Online resume requires the original project/entity and run ID; incompatible config/manifest or stale checkpoints are rejected. Offline resume starts a new offline segment with continuous local optimizer steps.

Normal and exception exits finalize W&B with distinct status, and Python exceptions are preserved locally. SIGKILL or power loss cannot run finalization. A recovery CLI replays local metrics/tables/charts into a clearly named new run. Writable local cache/staging defaults prevent artifact uploads from depending on a writable home cache. Offline serialization has been verified; authenticated cloud delivery remains to be checked on CIRCE.

10 Implementation evidence and limitations

The local suite passed 24 tests under Python 3.11.14, PyTorch 2.3.0, torchvision 0.18.0, NumPy 1.26.4, Hydra 1.3.2, W&B 0.22.3 and pymunk 6.6.0. Tests cover actual PushT rerendering, state/-mask integrity, all objectives and conditioning modes, backpropagation, CLS compatibility, CPU resume equality, probe fitting and offline W&B chart recovery. Python 3.9 syntax was checked; its actual runtime, CUDA, real CIRCE training data and DINO weight loading were not tested locally.

1. The state model is incomplete and static images do not identify every state component.

2. Reconstruction and MSE alignment do not guarantee causal disentanglement or prevent collapse.
3. The same finite offline noise/color samples recur throughout training.
4. Reset rejection and accepted-trajectory lengths can change the evaluated population.
5. Linear-probe success measures accessible information, not policy success.
6. The current strength diagnostics concern each corruption separately. Joint color-plus-noise generalization is a future experiment, not an implemented benchmark result.
7. Default loss weights, corruption ranges and sample budget require empirical validation.
8. Shape compatibility with AdaJEPa does not establish latent compatibility with existing checkpoints.

No success-rate values from previous planning runs should be presented as results of this new encoder experiment. The next defensible output is a completed, reproducible representation comparison with clearly labeled experimental uncertainty.

A Runnable command sequence

These are Bash commands in the repository on CIRCE. Replace the source/checkpoint paths with actual paths. They are derived from the implemented Hydra config.

```
SOURCE=/absolute/path/to/pusht_dataset
PAIRS="$PWD/data/tracka_pairs"

python generate_pusht_pairs.py --config-name tracka \
  dataset.source_path="$SOURCE" dataset.output_path="$PAIRS" \
  generation.preview_only=true

# After inspecting the exported contact sheets:
python generate_pusht_pairs.py --config-name tracka \
  dataset.source_path="$SOURCE" dataset.output_path="$PAIRS" \
  generation.preview_only=false generation.ranges_reviewed=true

# Engineering smoke run; no full-scale training:
python train_encoder.py --config-name tracka \
  dataset.output_path="$PAIRS" training.objective=robust_alternating \
  training.total_optimizer_updates=2 training.batch_size=2 \
  logging.log_every=1

# Planned primary matrix, executed sequentially when resources are allocated:
for SEED in 0 1 2; do
  for MODE in clean_mae render_only corruption_only robust_alternating; do
    python train_encoder.py --config-name tracka \
      dataset.output_path="$PAIRS" training.objective="$MODE" \
      training.seed="$SEED" logging.group="trackA-seed${SEED}"
  done
done
```

For each printed run directory, evaluate its final checkpoint and retain the results:

```
python eval_encoder_probes.py --config-name tracka \
  dataset.output_path="$PAIRS" evaluation.encoder=checkpoint \
  evaluation.checkpoint=/absolute/path/to/run/checkpoint_latest.pt
```

```
python eval_encoder_probes.py --config-name tracka \
    dataset.output_path="$PAIRS" evaluation.encoder=random

python eval_encoder_probes.py --config-name tracka \
    dataset.output_path="$PAIRS" evaluation.encoder=dino

python aggregate_tracka_results.py \
    /absolute/path/to/probe_clean /absolute/path/to/probe_robust \
    --output-dir "$PWD/tracka_outputs/comparison"

python recover_tracka_wandb.py /absolute/path/to/run
```

Use `logging.mode=offline` when cloud access is unavailable. Resume requires the original nondefault model/loss/training settings along with `training.resume=/absolute/path/to/checkpoint_latest.pt` and a larger total budget. Dataset source files include `states.pth`, `velocities.pth` when needed, `seq_lengths.pkl`, and optional `shapes.pkl`. Keep any real large-scale launch within the cluster’s allocated resources; no scheduler resource request is invented by these commands.

B Code-to-method map

File	Method responsibility
<code>conf/tracka.yaml</code>	Defaults and all experiment switches
<code>tracka/data.py</code>	Rerendering, rejection, four-view records, manifests and loading
<code>tracka/model.py</code>	Encoder, shared decoder, mask restoration and exact losses
<code>tracka/train.py</code>	Sampling, objective schedule, optimization and resume
<code>tracka/probes.py</code>	Clean-trained ridge probes, references and shifted evaluation
<code>tracka/logging.py</code>	Durable local logs and W&B lifecycle/charts
<code>planning/image_corruption.py</code>	Canonical and backward-compatible corruption functions
<code>aggregate_tracka_results.py</code>	Protocol-compatible grouping, CSV and seed SD figure
<code>tests/test_tracka.py</code>	Smoke and regression evidence
<code>docs/tracka_validation.md</code>	Actual validation environment and limitations

References

- [1] K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick. *Masked Autoencoders Are Scalable Vision Learners*. arXiv:2111.06377, 2021. <https://arxiv.org/abs/2111.06377>.
- [2] M. Oquab et al. *DINOv2: Learning Robust Visual Features without Supervision*. arXiv:2304.07193, 2023. <https://arxiv.org/abs/2304.07193>.